

SysMD Notebook Kickstart

Technische Universität Kaiserslautern

Fachbereich Informatik

Chair of Cyber-Physical Systems

1 Download, Installation, and Start	3
2 SysMD Notebook: The Tool	4
2.1 Start of SysMD Notebook and Settings	4
2.2 Create a new SysMD Document	5
2.3 Add, Delete, Edit Document-Elements	5
Add and Delete SysMD Document-Elements	5
Edit Document-Elements	6
2.4 Analysis and Results	7
3 The Language SysMD	7
3.1 Taxonomy of Classes (IsA-Relationship)	8
3.2 Features of Classes (hasA-Relationship)	9
3.3 Types and Functions in Expressions	10
Functions over Aggregations of Features / SUM on hasA relationship	11
Functions over Subclasses / bySubclasses-Function via isA Relationship	11
3.4 Inheritance	12
3.5 Interval-Semantics in Specifications: All or One?	13
4 “Hybrid Spaces” and SysML v2	13
4.1 Supported languages: Markdown, SysMD, SysML v2 Textual	14
4.2 SysMLv2 API	14
5 Known issues	14

1 Download, Installation, and Start

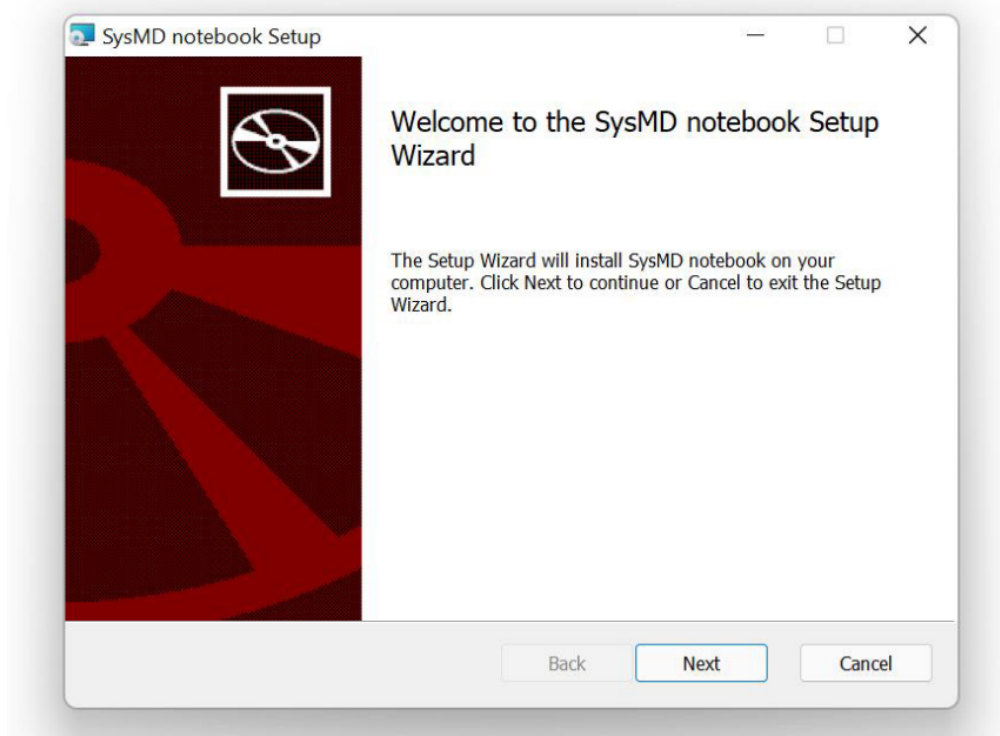
The SysMD Notebook installer can be downloaded under the following URL:

<https://owncloud.edacentrum.de/index.php/apps/files/?dir=/genial!/Verwertung/Tutorials/>

For installation, an installer is needed; which one depends on the operating system (OS):

OS	File
MS Windows	SysMDnotebook[versionsnummer]. msi
Mac OS X; M1 processor	SysMDnotebook[versionsnummer]. dmg
Intel processor	(auf Nachfrage)
Linux	(auf Nachfrage)

After downloading, start the installer. The installer starts a setup-wizard; the following screenshots show the Windows installer.



Depending on your windows version and security settings, there might be requests from Windows.

Der Computer wurde durch Windows geschützt

Von Microsoft Defender SmartScreen wurde der Start einer unbekannten App verhindert. Die Ausführung dieser App stellt u. U. ein Risiko für den PC dar.
[Weitere Informationen](#)

Here, you have to choose “Further information”. Then, continue with “OK” and “Next” until SysMD Notebook is installed.

2 SysMD Notebook: The Tool

In Section 2 we give an overview of the tool “SysMD Notebook” without going into details of the language or modeling. .

2.1 Start of SysMD Notebook and Settings

After installation with the setup wizard, there are several options to start SysMD Notebook:

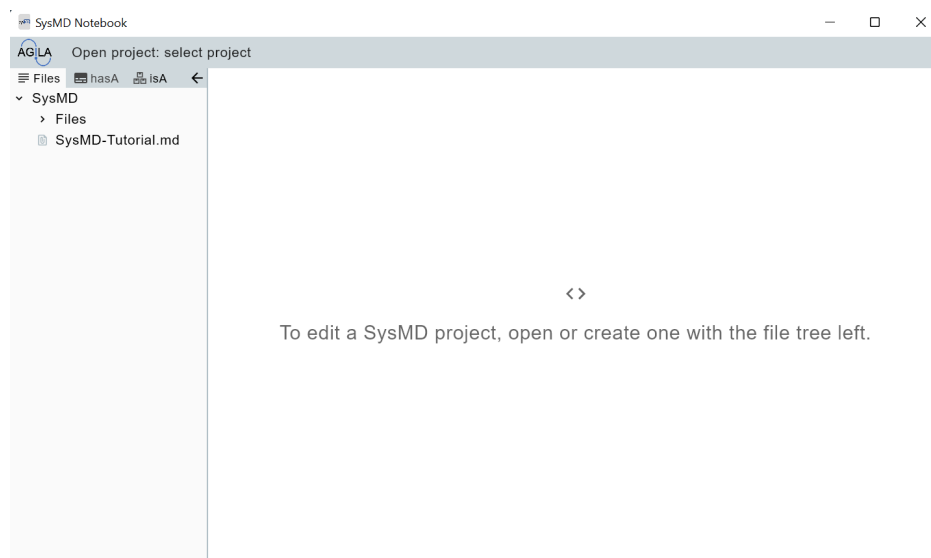
- There is an Icon on the Desktop (if you have selected that option),
- The application “SysMD Notebook” is in the folder with all your other applications.



*During the first run, SysMD creates a folder “**My Files/SysMD**”.*

This folder is used to save your data and also provides a copy of this document.

After starting the SysMD Notebook, the following Window opens:

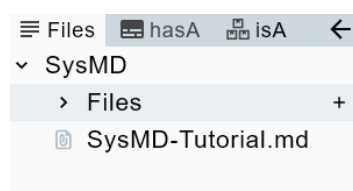


Left, we have a treeview. It allows us to

- 1) Select and load SysMD files.
- 2) Browse through the internal model and results; more on that later.

Right, we can edit our documentation and models.

2.2 Create a new SysMD Document



To create a new SysMD document, we must first create a file in which it will be saved. For this purpose, we select the treeview “Files” and navigate to the small “+” (see screenshot). By clicking on the small “+” we create a new SysMD Document that is, once saved, in a Markdown-File. After clicking on “+”, SysMD

Notebook asks for a filename. Enter a name with the extension “.md” and confirm with “Return”. The new file is then, once saved, in the folder “SysMD” in the home folder.

2.3 Add, Delete, Edit Document-Elements

A SysMD Document (File) is structured in *Elements*. An Element can be of the kinds

- *Documentation*. Documentation is written in the Markdown Markup-Language that is inbetween a What-You-See-Is-What-You get Editor like Word or Excel and pure text. Markdown allows us to document our model in a structured way, including a document structure (Chapters, Sections, etc.), Pictures, Tables. This helps us to better explain how we came to requirements – and others to understand it as well. In SysMD, Documentation is tightly interwoven and linked (via Relationships) with more formal, textual models.
- *Textual model*. We can formalize models the modeling languages SysMD oder einem Subset or a subset of SysML v2 textual. Both build on top of SysML’s KerML metamodel classes and can be exchanged via the SysMLv2 API.

Add and Delete SysMD Document-Elements

A new SysMD document is empty in the beginning. To add an element, we click on the small gray circle that is shown. It adds a document-element. The same small circle and the space between existing ones allows us also to insert new elements between existing ones:



Now we have already created our first document-element.

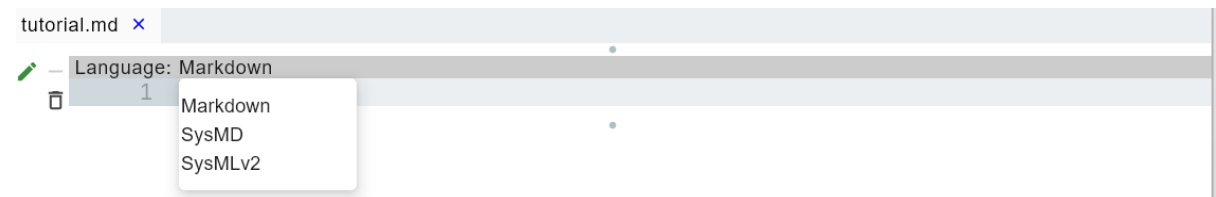


Left of each document-element, icons are shown. If an element is not selected, they are grey; if an element is selected, the icons are in different colors. If the mouse is over an icon, an explanation is shown what can be

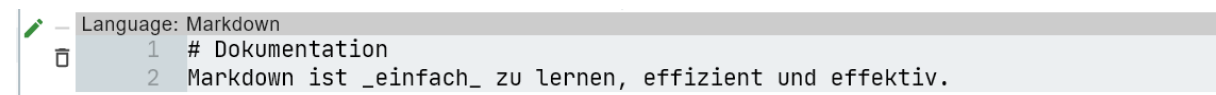
done. To delete an element, select the trash bin, to edit the pencil, and to minimize the document-element, select the “-”.

Edit Document-Elements

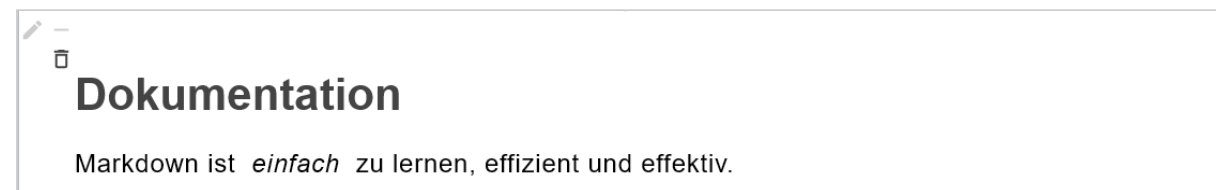
To edit a document-element, select the small pencil left of the element. Or simply double click on the element. Now we can edit the text of the element. By the menu on top of an active, selected element, we select the language of the document-element. Then we get a menu in which we can choose the language:



For a **Documentation**, we choose “Markdown” and write some text, e.g.:

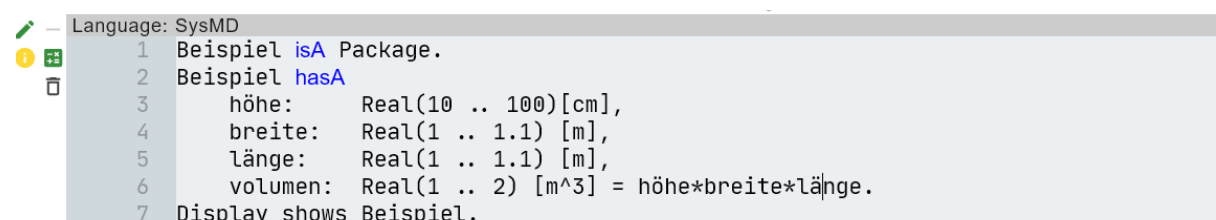
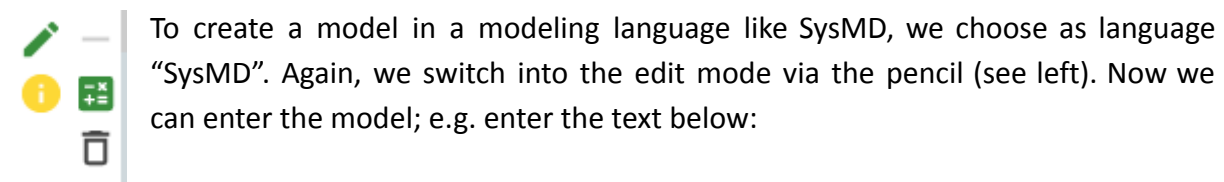


To render the element, we de-select it via clicking again on the pencil; then, we leave the edit-mode and the element is rendered by SysMD-Notebook:



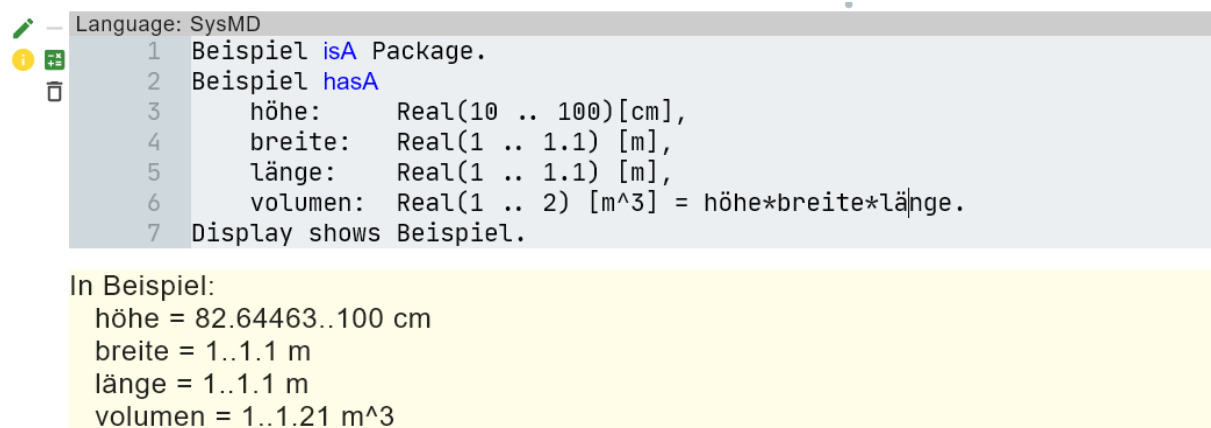
The text-markup language Markdown allows us to structure documents with headings and subheading, but as well tables, pictures and much more. For an introduction to Markdown we refer to:

<https://commonmark.org/help/tutorial/>



2.4 Analysis and Results

To analyze the textual model, we click on the calculator. This shows us eventually some syntactic or semantic errors; but not with the model above. To display the results, we select the yellow “i”. Then, SysMD shows us the feasible valuations of the variables for which all constraints can be satisfied:



The screenshot shows the SysMD interface. On the left, there is a toolbar with icons for editing (pencil), information (yellow 'i'), calculation (calculator), and deletion (trash). The main area displays a textual model in SysMD language. The model consists of seven lines: 1. 'Beispiel isA Package.', 2. 'Beispiel hasA', 3. 'höhe: Real(10 .. 100) [cm],', 4. 'breite: Real(1 .. 1.1) [m],', 5. 'länge: Real(1 .. 1.1) [m],', 6. 'volumen: Real(1 .. 2) [m^3] = höhe*breite*länge.', and 7. 'Display shows Beispiel.'. Below the model, a yellow box displays the feasible valuations for the variables: 'In Beispiel:', 'höhe = 82.64463..100 cm', 'breite = 1..1.1 m', 'länge = 1..1.1 m', and 'volumen = 1..1.21 m^3'.

```
Language: SysMD
1 Beispiel isA Package.
2 Beispiel hasA
3   höhe:      Real(10 .. 100) [cm],
4   breite:    Real(1 .. 1.1) [m],
5   länge:     Real(1 .. 1.1) [m],
6   volumen:   Real(1 .. 2) [m^3] = höhe*breite*länge.
7 Display shows Beispiel.
```

In Beispiel:
höhe = 82.64463..100 cm
breite = 1..1.1 m
länge = 1..1.1 m
volumen = 1..1.21 m^3

If we prefer to use “SysML v2 textual” as a modeling language, we select as language “SysML”.

3 The Language SysMD

In this section, we describe the language SysMD. The primary design objective of SysMD was to create a language that integrates into documents and that can be used by engineers, management, and domain experts directly:

Readability: Statements read like natural language sentences. Everybody shall be able to read and understand a SysMD model. No magic symbols or operators.

Steep learning curve: To write models, the number of syntactic rules was minimized; existing rules are applied in the same form for different purposes. A grammar that fits on a single sheet.

Clarity: The specification of requirements is syntactically separated from modeling of dependencies and computations. Requirements can be seen at the first glance.

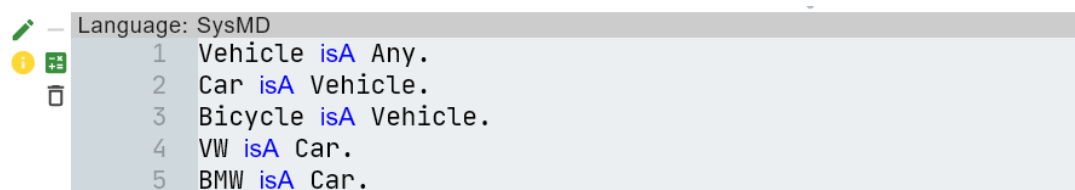
Interoperability: SysMD Notebook leans towards SysMLv2: lexical conventions, name resolution, metamodel follow the SysMLv2 standard. Tool coupling and exchange of data is supported via the SysMLv2 API. But note, that the standard is not finished and we do not support the complete standard.

3.1 Taxonomy of Classes (IsA-Relationship)

For modeling a domain we typically start with a taxonomy that explains the different classes that we find in a domain. For example, if we model Vehicles, we can classify different kinds of vehicles. SysMD taxonomies are trees (with a node as root, internal nodes, and leaves as usual in computer science). In SysMD, we hence always have a single root: the node “Any(thing)”. Classes of the domains are then broken down to the most specialized classes that are the leaves of the taxonomy:

- The root node (Any) is the most general class
- Towards the leaves, classes become more and more specific.

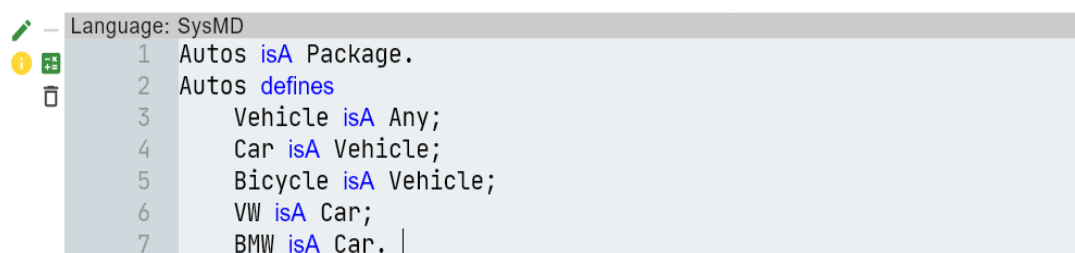
To model the taxonomy, we use the relationship “is a”. In SysMD this is done by using the Keyword “isA”. Hence, we can model a taxonomy for vehicles in SysMD as follows:



```
Language: SysMD
1 Vehicle isA Any.
2 Car isA Vehicle.
3 Bicycle isA Vehicle.
4 VW isA Car.
5 BMW isA Car.
```

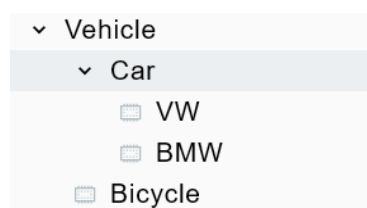
This is nice, but with a growing number of classes and domains we will lose the overview. We need to organize our class definitions *hierarchically*. For this purpose we use a “Package”. A package separates its contents from the rest; definitions are directly visible only inside a package. Or, we can specify the “path” towards a definition, where we separate the names by two double points (“::”); we will see examples later. Note: for the syntax of paths and the visibility of names (“name resolution”), SysMD uses equal definitions as SysMLv2.

Syntactically the use of a package and the definitions are shown below. Note, that the definition-statements that belong to a common package are separated by semicolons. The enumeration of statements ends with a dot:



```
Language: SysMD
1 Autos isA Package.
2 Autos defines
3     Vehicle isA Any;
4     Car isA Vehicle;
5     Bicycle isA Vehicle;
6     VW isA Car;
7     BMW isA Car. |
```

After analysis (green calculator, left of element), the respective definitions are added to the internal model. In the “tree-view” under the relationship “isA” they are then displayed; eventually, closing/opening it is needed to get a refresh:





We use taxonomies to structure our domain. Objective is to introduce classes and their specializations that model different variants. Later, we compare different variants in different contexts. And see that inheritance makes taxonomies helpful.

3.2 Features of Classes (hasA-Relationship)

Once we have a taxonomy with different classes, we describe what formally differentiates the classes. For this purpose, we describe features. In SysMD, we use for this purpose the “hasA” relationship.

Assume, we want to describe features of our Vehicle classes. We then create a package of vehicle parts and add e.g. an engine and wheels to it. Then, we can explain what differentiates a bicycle from a car. Reminder: we can access the elements of this package from the package of vehicles via its path as shown in the example below.

```
Language: SysMD
1 Autoteile isA Package.
2 Autoteile defines
3   Engine isA Component;
4   Wheels isA Component.
5
6 Autos::Car hasA
7   wheels: [4 .. 4] Autoteile::Wheels,
8   engine: [1 .. 2] Autoteile::Engine,
9   mass: Real (100 .. 1000) [kg].
10
```

As can be seen, we can now make statements about different features of different kinds of vehicles by listing them as a feature using the *hasA* relationship. Generally, each feature is listed with:

- Name (i.e.. “wheels”), followed by a double point,
- Number (multiplicity), that is a range of Integers like (i.e. [4..4]),
- Class, that is what class describes the feature (i.e. “Autoteile::Wheels”).

Besides a decomposition into further elements, we can also model some numerical or Boolean properties. They may also have a physical unit. Then, we specify:

- Name (i.e.. mass)
- The specified type and range (i.e.. Real (100 .. 100))
- The required unit (i.e.. [kg])

```
Language: SysMD
1 SiEinheitenBeispiel isA Package.
2
3 SiEinheitenBeispiel hasA
4   t: Real [s] = 1.0 [s],
5   v: Real [m/s] = 3.0 [m/s],
6   g: Real [m/s^2] = 4.0 [m/s^2],
7   s: Real [m/s] = sqrt(sqr(v)+sqr(g)*sqr(t)).
8
9 Display shows SiEinheitenBeispiel.
```

In SiEinheitenBeispiel:

```
t = 1..1 s
v = 3..3 m / s
g = 4..4 m / s^2
s = 5..5 m / s
```

Even more, the concrete values of a property or the multiplicity of a feature can be constrained by dependencies; then, SysMD notebook computes the feasible values by considering all constraints.

SysMD supports all SI units, derived units, many national unit systems, per-cent notations, but also computation with logarithmic units (Dezibel). Units are converted automatically before computations are done, and consistency of units in equations is checked: the unit left

```
Language: SysMD
1 DateTime isA Package.
2
3 DateTime hasA
4   date: Real[DateTime] = DateTime("2021-10-10T03:00:00"),
5   time: Real[a] = 1.0 a,
6   dateResult: Real[DateTime] = date + time.
7
8 Display shows DateTime.
```

```
In DateTime:
date = 2021-10-10T03:00
time = 1..1 a
dateResult = 2022-10-10T03:00
```

of a dependency, and the unit right of it must be convertible into each other.

For data and time the ISO format is supported. We can add and subtract times in this format.

3.3 Types and Functions in Expressions

The following data types are supported:

- Real (Intervals)
- Integer (Intervals) – Not for IRIS.
- Boolean
- String – Not for IRIS.

In SysMD expressions, the following functions can be used (v2.5.20+)

- *ceil* - round Real x to next higher Integer – Not for IRIS.
- *floor(x)* - round Real x to next lower Integer – Not for IRIS.
- *exp(x)* - exponential function of x
- *log(x)* - natural logarithm of x
- *power2* – Not for IRIS.
- *powerb* – Not for IRIS.
- *sqr(x)* - square of x
- *sqrt(x)* - square root of x
- *linear(a, b, c, d, ...)* – linear interpolation through pairs of values specifying (x, y).
- *ITE(condition, if, else)* – ITE function; if Bedingung then if-value, else then-value
- *sum_i(...)* Iteration over i – Not for IRIS.
- *not(x)*
- *a and b*
- *a or b*
- Additional functions for querying subclasses (*isA*) and features (*hasA*).

Functions over Aggregations of Features / SUM on hasA relationship

Let's extend our library of car parts and see what the solver in SysMD can do. Assume, we are interested in the overall mass of different configurations of features - and which combinations satisfy some constraints.

```
Language: SysMD
1 Autoteile isA Package.
2 Autoteile defines
3   Body isA Component;
4   Engine isA Component;
5   Wheel isA Component.
6
7 Autoteile::Engine hasA mass: Real(200.0) [kg].
8 Autoteile::Wheel hasA mass: Real(50.0) [kg].
9 Autoteile::Body hasA mass: Real(100.0) [kg].
10
11 Autos::Car hasA
12   body: Autoteile::Body,
13   wheels: [4 .. 4] Autoteile::Wheel,
14   engine: [1 .. 2] Autoteile::Engine,
15   mass: Real [kg] = SUM(mass).
16
17 Display shows Autos::Car::mass.
```

mass = 500..700 kg

Often, properties of a system are defined by its parts. A simple example is the mass. In the example left we use the function SUM(mass) for this purpose: The mass of a Vehicle is the mass of all of its parts.

SUM computes the sum of a property given as a parameter over all features, considering its multiplicity. If no property is given, it continues in the features of a feature, and so

on (transitivity).

Functions over Subclasses / bySubclasses-Function via isA Relationship

For some classes its properties might be clear; e.g. for bicycles we can guess its mass, or for cars as well. But it can be difficult the more abstract we are in the taxonomy. What is the possible mass of arbitrary vehicles? SysMD can help by the function *bySubclasses(name)*.

In the Vehicle library, we can hence find out the mass of all kind of vehicles as follows:

```
Language: SysMD
1
2 Autos::Bicycle hasA mass: Real(10 .. 20) [kg].
3 Autos::Vehicle hasA mass: Real = bySubclasses(mass).
4 Display shows Autos::Vehicle::mass.
```

mass = 10..700

However note, that this does not give any constraints or specifications; it just computes the possible mass via subclasses. And this intention is shown in the textual model by having no specific constraint besides the fact that it is some Real-valued quantity.

3.4 Inheritance

As of now, we skipped one important thing: inheritance. In taxonomies, SysMD applies inheritance to reduce the modeling effort, and the Liskov principle to have sound semantics when reasoning about possible configurations. What does the Liskov-Principle mean? Simple:



“A specialization can always replace its superclass”.

What is its impact? If we ask for a vehicle, all of its subclasses (i.e. bicycle, car, and subclasses thereof) would satisfy our modeled needs. In other words, if we specify that we want a class, all of its subclasses are possible variants. In our example, variants or possible solutions would be:

- Bicycle
- Car, including its specializations
 - VW
 - BMW

Of course, this is only possible if the Liskov principle holds. To support this, the semantics of inheritance of SysMD strictly follow this principle and when we define a subclass, it inherits everything from the superclass such that it is fulfilled.

Note, that this does not mean that subclasses must be similar to its superclasses. Subclasses can have additional features, but not limitations. This is checked by the SysMD solver.

Obviously, a VW and a BMW can differ from a generic “Car”. But only in a way such that the Liskov principle holds. Assume, we model the power of Cars and its subclasses as follows:

```
Language: SysMD
1 Autos::Car hasA power: Real(10 .. 1000) [kW].
2 Autos::VW hasA power: Real(20 .. 1100) [kW].
3 Autos::BMW hasA power: Real(150 .. 400) [kW].
```

These specifications are consistent with the Liskov principle: If one wants a car with a power from 10 to 1000 kW, a VW or BMW will satisfy this constraint. However, if we change the specification of VW to a power of max. 1100 kW, this is in contradiction to line 1 which says that all vehicles have a power in the range of 10 to 100 kW. Hence, SysMD notebook displays an error:

```
Language: SysMD
1 Autos::Car hasA power: Real(10 .. 1000) [kW].
2 Autos::VW hasA power: Real(20 .. 1100) [kW].
3 Autos::BMW hasA power: Real(150 .. 400) [kW].
```

ERROR: In power: column 29: INCONSISTENCY: subclass value 20..1100 of power must be refinement of superclass value 10..1000

If a superclass is only used for structuring a domain, but not for setting constraints, there should be no specified value; eventually, we can give a very broad range to find unreasonable values in subclasses. Alternatively, we can use the function *bySubclasses*.

3.5 Interval-Semantics in Specifications: All or One?

Intervals in specifications can have different semantics. In the previous examples, we used the semantics that we want to pick a concrete value from a range: a value e.g. for the power of a car as above. By specifying

```
power: Real( 20 .. 100) [kW]
```

we mean that we want a vehicle with ONE value of power (in kW). A vehicle with 10 kW would not satisfy our requirement.

However, the same semantics is wrong if we need a vehicle that can be used in the temperature range of -20 to 60 °C. Then, the specification by

```
temperature: Real( -20 .. 60) [°C]
```

is simply wrong: it means that we want a car that works in some or one temperatur from the specified range. Hence, a vehicle that works in a temperature range from 0 to 40 °C would (incorrectly) satisfy this requirement. SysMD uses the keyword “all” to specify the needed semantics for the given case. It means that the constraint must be satisfied for all possible values. Hence, the correct model in SysMD is:

```
temperature: all Real( -20 .. 60) [°C]
```

SysMD considers these semantics in

- Constraint-Propagation and
- Inheritance

4 “Hybrid Spaces” and SysML v2

When requirements and specifications are exchanged, common standards, tools, and languages simplify communication. However, in heterogeneous groups of stakeholders, this can be a hurdle if different stakeholders use different conventions and have different terminologies.

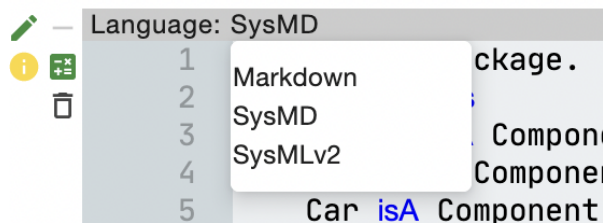
SysMD deals with this issue by supporting the concept of “hybrid spaces”. This means that models can be presented and edited in different ways, but are internally converted to a unified model. SysMD Notebook and AGILA represent its inputs by a (subset of) KerML metamodel, which is the “internal model” specified by the upcoming OMG standard SysMLv2.

4.1 Supported languages: Markdown, SysMD, SysML v2 Textual

In SysMD notebook, we can use the languages

- Markdown for documentation and elicitation of requirements,
- SysMD for moving modeling requirements and checking its consistency,
- SysML v2 Textual subbed for modeling requirements in a more formal way than in SysMD, addressing the stakeholders that are more familiar with systems engineering.
- *Other languages and interfaces are planned.*

The language can be selected in the pulldown menu of each element.



A simple comparison of SysML v2 Textual can be seen below (note, that the syntax of SysMLv2 textual might have changed; the standard is still at draft level).

```
Wheel {  
  attribute mass: ScalarValue::Real = 70@[SI::kg];  
  // model constraint & unit of mass, ...  
}  
  
Car :> Vehicle {  
  part Wheel [4 .. 8];  
  attribute mass = ... // model constraint, unit, ...  
}
```

```
Car isA Vehicle.  
  
Car hasA  
  wheel: [4 .. 8] Wheel,  
  mass: Real(100..1000)kg = sumHasA(mass).  
  
Wheel hasA  
  mass: all Real(50 .. 100) kg.
```

it must be noted that the supported subset of SysML v2 textual is small and that the standard is still evolving.

4.2 SysMLv2 API

The backend of SysMD Notebook leans towards the API specified in OMG's SysML v2. While all endpoints are available, the DAO entities might be different; however, adaptations of the DAO can be made easily by "SysMD experts" to achieve interoperability with other tools.

5 Known issues

- SysMD notebook does not display correctly on a 2nd screen.
- The supported subset of SysMLv2 is small and the standard is not final.